Mini-projeto 1: buscas clássicas e otimização

Eduardo N. S. R.

PPGCO / PGC112 – Faculdade de Computação (FACOM)

Repositório Github: vndmtrx/ppgco-pgc112-ia

Uberlândia – MG, Brasil, 2026

1 Introdução

Este relatório apresenta a resolução do Mini-Projeto 1 sob duas abordagens da Inteligência Artificial: (i) busca clássica no espaço de estados discreto (BFS e Algoritmo Guloso) e (ii) otimização contínua no hipercubo fuzzy via Gradiente Descendente com regularização por custo de vizinhança.

O problema consiste em transformar a fita inicial canônica de $L = 23$ bits:

$$s_0 = \texttt{00001001000011001100111} \quad (1)$$

na fita alvo composta exclusivamente por zeros:

$$s_{\text{alvo}} = \texttt{00000000000000000000000} \quad (2)$$

O modelo de custos estabelece que: (i) o flip de 1 bit possui custo unitário 1; e (ii) o flip de um bloco de n bits consecutivos possui custo uniforme 1. Essa regra confere uma economia de escala crítica, na qual inverter agrupamentos adjacentes custa o mesmo que uma modificação isolada.

O código-fonte, a suíte de testes e os benchmarks estão na pasta *mini_projeto_1* dentro do repositório Git `vndmtrx/ppgco-pgc112-ia`, cujo `README.md` detalha a instalação e execução em ambiente isolado (*virtualenv*).

2 Parte I: Buscas Clássicas

2.1 Subrotinas para Flips e Álgebra Bitwise

Para eliminar a sobrecarga de memória do CPython associada à instanciação de strings e objetos no heap durante a geração de milhões de estados, a fita foi representada diretamente como um número inteiro nativo (`int`).

As operações de inversão foram executadas diretamente via máscaras binárias com XOR ($\oplus$):

- **Flip de 1 bit:** estado $\oplus$ $(1 \ll \text{pos})$, para $0 \leq \text{pos} < L$.
- **Flip de bloco contíguo:** estado $\oplus$ $(((1 \ll n) - 1) \ll \text{pos})$, para $2 \leq n \leq L - \text{pos}$.

Adotou-se a ordenação *little-endian* (bit 0 como o menos significativo, 2^0). Essa representação viabiliza verificar estados visitados com hash $O(1)$ e executar centenas de milhares de flips por segundo.

2.2 Algoritmo BFS e Solução Ótima

Como todas as transições possuem custo uniforme unitário ($c = 1$), a primeira solução gerada pela Busca em Largura (BFS) possui garantia formal de otimalidade. O algoritmo foi implementado com fila FIFO

(`deque`) e conjunto de visitados (`set`), realizando o teste de objetivo antecipado na geração do nó.

Para a sequência canônica de 23 bits, o algoritmo retornou o **custo ótimo igual a 5** e a seguinte sequência de operações:

$$\mathcal{S}_{\text{BFS}} = [(\texttt{'bit'}, 15), (\texttt{'bit'}, 18), (\texttt{'bloco'}, 3, 2), (\texttt{'bloco'}, 7, 2), (\texttt{'bloco'}, 0, 11)]$$

onde $(\texttt{'bloco'}, \text{pos}, n)$ inverte n bits consecutivos a partir de pos, e $(\texttt{'bit'}, \text{pos})$ inverte a coordenada individual. Para alcançar essa profundidade $d = 5$, o BFS expandiu **752.059 nós**, visitou **2.745.489 estados** e consumiu **35,42 s**. O comportamento de crescimento de nós do algoritmo é ilustrado na Figura 1.

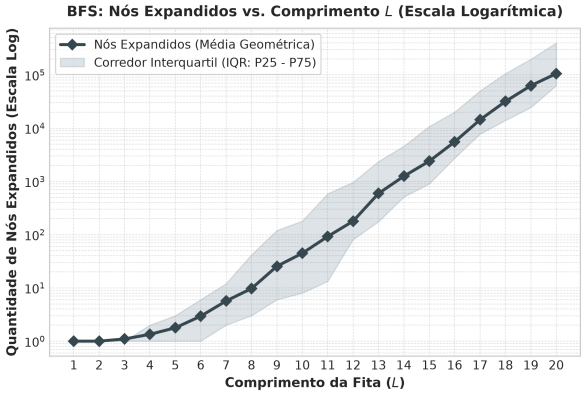


Figura 1 – Crescimento exponencial do número de nós expandidos pelo BFS em função de L no pior caso teórico.

2.3 Algoritmo Guloso e Heurísticas

A Busca Gulosa de Melhor Escolha utiliza uma função heurística $h(s)$ para priorizar estados com menor custo estimado até o alvo:

- **Distância de Hamming (h_1):** Conta o número absoluto de bits 1 ($h_1(s_0) = 9$). Como flips de bloco eliminam múltiplos 1s sob custo unitário, h_1 superestima o custo real ($9 > 5$), violando a admissibilidade ($h(s) \leq h^*(s)$).
- **Contagem de Blocos Contíguos (h_2):** Avalia quantos agrupamentos contíguos de 1s existem na fita ($h_2(s_0) = 5$). Como cada bloco conexo pode ser extinto em uma única operação de custo 1, h_2 coincide com o custo real ótimo ($h_2(s) = h^*(s)$), garantindo monotonicidade estrita.

2.3.0.1 Poda de Ciclos.

Devido à miopia do Guloso, sem memória histórica o algoritmo pode alternar flips reversíveis e entrar em **loops infinitos**. Com a inclusão da tabela hash de visitados (*poda de ciclos*), a busca gulosa com h_2 retornou o **custo total igual a 5** e a sequência de transformações:

$$\mathcal{S}_{\text{Guloso}} = [(\texttt{'bloco'}, 0, 3), (\texttt{'bloco'}, 5, 2), (\texttt{'bloco'}, 9, 2), (\texttt{'bit'}, 15), (\texttt{'bit'}, 18)]$$

atingindo o alvo ótimo em exatamente **5 expansões** e tempo de **0,16 ms**. A Figura 2 ilustra a eficácia da poda na contenção do tempo de processamento.

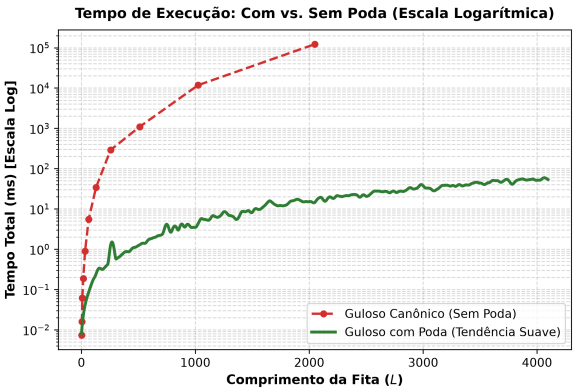


Figura 2 – Comparativo empírico de tempo na Busca Gulosa com e sem poda de ciclos.

2.4 Análise de Desempenho e Escalabilidade ($L = 2$ a 4096)

O número de intervalos contíguos $[i, j]$ para uma fita de tamanho L define o fator de

ramificação exato da árvore de busca, dado pela soma das L inversões unitárias com as combinações de blocos: $b(L) = L(L+1)/2 = \Theta(L^2)$. Para a fita de $L = 23$, cada nó gera $b = 276$ sucessores, governando a complexidade espacial do BFS em $O(b^d)$.

2.4.0.1 Metodologia de Pior Caso por Faixa de Bits.

Para avaliar o limite superior (*upper bound*) de consumo de recursos e determinar o tamanho máximo das árvores geradas em cada faixa de bits, os experimentos basearam-se no **pior caso teórico**: fitas com bits alternados (`'010101...'`). Essa configuração maximiza os blocos unitários e empurra a profundidade para o diâmetro máximo do grafo, $d_{\max} = \lceil L/2 \rceil$.

Sob esse estresse máximo, a quantidade de nós expandidos explode exponencialmente (Figura 1): enquanto para $L = 4$ o BFS expande 1,5 nós e para $L = 10$ são 153 nós (0,54 ms), em $L = 16$ expandem-se 13.512 nós e para $L = 20$ são necessários 263.922 nós (2,96 s). Para $L > 20$, o pior caso exige varrer quase todo o espaço de estados ($2^{23} \approx 8{,}38 \times 10^6$ nós para $L = 23$), esgotando a memória RAM disponível. Os gráficos das Figuras 1 e 2 têm como base essa análise limite.

2.4.0.2 Validação na Fita Oficial e Limite $L = 4096$.

Em contrapartida, na fita canônica de 23 bits proposta pelo docente, os 1s agrupam-se em apenas 5 blocos, permitindo ao BFS encontrar o ótimo em profundidade $d = 5$ (752.059 nós e 35,42 s). Já o Guloso com poda de ciclos, além de resolver a fita oficial em 0,16 ms, manteve tempo de execução sub-segundo até $L = 4096$ (Tabela 1): mesmo sob fator de ramificação $b(4096) \approx 8{,}39 \times 10^6$ sucessores por nó, convergiu em apenas 48,56 ms.

Tabela 1 – Desempenho da Busca Gulosa com Poda (h_2) até $L = 4096$.

L	Fator $b(L)$	Custo Médio	Nós Expandidos	Tempo (ms)
4	10	1,0	1,0	0,003
16	136	4,0	4,0	0,023
64	2.080	5,0	5,0	0,097
256	32.896	10,0	10,0	1,189
1024	524.800	12,0	12,0	4,643
4096	8.390.656	12,0	12,0	48,563

3 Parte II: Otimização Contínua com Gradiente Descendente

3.1 Mapeamento Fuzzy e Espaço de Soluções

Mapear a sequência de bits para um escalar inteiro $N \in \mathbb{N}$ geraria descontinuidades severas na reta real: inverter o bit mais significativo produziria saltos de amplitude 2^{L-1}, induzindo falsas divergências no gradiente.

Pelo relaxamento *fuzzy*, o k-ésimo bit da fita torna-se uma variável contínua independente:

$$x_k \in [0, 1], \quad \forall k \in \{1, 2, \ldots, L\} \tag{3}$$

A fita passa a descrever um vetor $\mathbf{x} \in \mathbb{R}^L$ em um hipercubo unitário, onde o alvo nulo corresponde à origem $\mathbf{y} = \mathbf{0}$. O objetivo consiste em encontrar uma trajetória contínua que altere x_k penalizando alterações não-consecutivas.

3.2 Função Erro $\mathcal{L}(\mathbf{x})$ e Relação com as Transformações

A função de erro quadrático é definida por:

$$\mathcal{L}(\mathbf{x}) = \sum_{k=1}^{L} (x_k - y_k)^2 \tag{4}$$

Para o alvo nulo $\mathbf{y} = \mathbf{0}$, $\mathcal{L}(\mathbf{x}) = \sum_{k=1}^{L} x_k^2$. Nos vértices booleanos $\{0, 1\}^L$, $\mathcal{L}(\mathbf{x})$ coincide exatamente com a Distância de Hamming $h_1(\mathbf{x})$. Minimizar $\mathcal{L}(\mathbf{x})$ atrai continuamente cada coordenada para zero, e o vetor deslocamento entre iterações consecutivas $\Delta\mathbf{x}^{(t)} = \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)} = -\eta \nabla \mathcal{L}(\mathbf{x}^{(t)})$ re-

presenta a transformação contínua instantânea aplicada à fita.

3.3 Gradiente Descendente sem Custo e Comportamento de η

A regra clássica de atualização é dada por:

$$x_i \leftarrow x_i - \eta \frac{\partial \mathcal{L}}{\partial x_i} = x_i - 2\eta(x_i - y_i) \qquad (5)$$

Para $y_i = 0$, a dinâmica analítica desacoplada é:

$$x_i^{(t+1)} = (1 - 2\eta) x_i^{(t)} \qquad (6)$$

Essa recorrência expressa formalmente três regimes de operação:

- **Subcrítico ($\eta < 0{,}5$):** O fator de contração permanece positivo e menor que 1 (por exemplo, 0,8 para $\eta = 0{,}1$), produzindo decaimento exponencial suave e monótono em direção ao estado nulo.
- **Crítico Unitário ($\eta = 0{,}5$):** O fator anula-se identicamente, zerando todo o resíduo da função erro em **um único passo exato**.
- **Violação de Domínio e Divergência ($\eta > 0{,}5$):** O passo ultrapassa o mínimo global e projeta o bit para valores negativos (atingindo $-0{,}5$ para $\eta = 0{,}75$), violando o domínio fuzzy $[0, 1]$. Para $\eta = 1{,}0$, o bit oscila perpetuamente entre $+1$ e -1 sem convergir, e para $\eta > 1{,}0$, o erro diverge de forma ilimitada.

3.3.0.1 Sequência de Transformações.

Os vetores diferença entre passos consecutivos $\Delta \mathbf{x}^{(t)} = \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}$ representam a sequência de transformações do algoritmo contínuo. Para $\eta = 0{,}5$, a convergência ocorre em 1 único passo exato com o vetor $\Delta \mathbf{x}^{(0)} = -\mathbf{x}_0$, aplicando -1 pontualmente nas nove posições ativas. Para $\eta = 0{,}1$, obtém-se uma sequência de transformações amortecidas que acumulam assintoticamente o cancelamento total do vetor inicial.

A Figura 3 ilustra as curvas de nível da perda quadrática e as trajetórias geradas para cada regime de η, evidenciando a convergência imediata para $\eta = 0{,}5$, a oscilação com valores negativos para $\eta = 0{,}75$ e a divergência explosiva para $\eta = 1{,}05$.

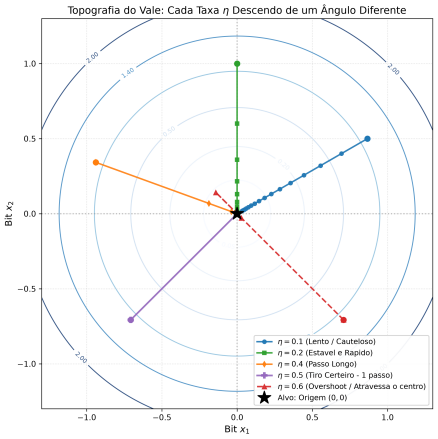


Figura 3 – Curvas de nível de $\mathcal{L}(\mathbf{x})$ e trajetórias 2D do Gradiente Descendente para múltiplos valores de η.

3.4 Inclusão da Função Custo $C(z)$ e Regra da Cadeia

Para favorecer inversões adjacentes e penalizar mutações isoladas, o vetor z modela a intensidade da transformação através do gradiente: $z_i = |\nabla_i \mathcal{L}(\mathbf{x})| = 2|x_i|$ (ou suave $z_i = x_i$). Definimos a função custo contígua via diferença quadrática entre vizinhos adjacentes:

$$C(\mathbf{x}) = \sum_{k=1}^{L-1} (x_{k+1} - x_k)^2 \qquad (7)$$

Essa função anula-se quando coordenadas vizinhas variam em uníssono (blocos uniformes) e atinge valores máximos em alternâncias de alta frequência.

Considerando a perda regularizada $J(\mathbf{x}) = \mathcal{L}(\mathbf{x}) + \lambda C(\mathbf{x})$, aplicamos a regra da cadeia para uma coordenada interna x_i

$(1 < i < L)$:

$$\begin{aligned}\frac{\partial J}{\partial x_i} &= \frac{\partial \mathcal{L}}{\partial x_i} + \lambda \frac{\partial C}{\partial x_i} \\ &= 2x_i + 2\lambda(2x_i - x_{i-1} - x_{i+1})\end{aligned} \tag{8}$$

Demonstração da influência adjacente: O termo $2\lambda(2x_i - x_{i-1} - x_{i+1})$ evidencia que a taxa de variação de x_i é diretamente governada pelas coordenadas dos vizinhos imediatos x_{i-1} e x_{i+1}.

3.5 Minimização para Diferentes Valores de λ e Limite de Estabilidade

A equação do gradiente revela comportamentos distintos conforme a configuração local de vizinhança:

- **Interior do Bloco $(1,1,1)$:** A simetria entre as coordenadas adjacentes anula a penalidade de regularização, conduzindo a atualização unicamente pelo gradiente da função erro $(2x_i)$.
- **Borda do Bloco $(0,1,1)$:** A descontinuidade lateral gera uma força de atração proporcional a 2λ, retardando a modificação prematura da extremidade.
- **Bit Isolado $(0,1,0)$:** Com vizinhos nulos, a penalidade atinge intensidade máxima e amplifica o gradiente efetivo para $2x_i + 4\lambda$. Esse ganho acelera a eliminação do bit solitário como ruído de alta frequência antes da remoção dos blocos contíguos.

3.5.0.1 Cota Crítica de Estabilidade.

A derivada segunda de J em relação a cada coordenada é $\frac{\partial^2 J}{\partial x_i^2} = 2+4\lambda$. Valores elevados de λ aumentam a curvatura da função de custo. Para evitar oscilações divergentes, a taxa de aprendizado deve respeitar o limite:

$$\eta \leq \eta_{\text{crit}} \approx \frac{1}{2+4\lambda} \tag{9}$$

Caso η ultrapasse essa cota, as atualizações consecutivas amplificam o erro em vez de reduzi-lo, resultando em divergência numérica $(J \to \infty)$. Para $\lambda = 20{,}0$, a restrição impõe $\eta \leq 0{,}0122$. O diagrama de fase empírico confrontado com a curva teórica é apresentado na Figura 4.

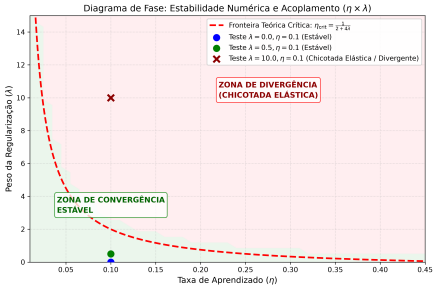


Figura 4 – Diagrama de fase de estabilidade numérica no plano (η, λ) e curva limite teórica de estabilidade.

4 Considerações Finais

A formulação do trabalho demonstra a dualidade entre espaços de busca em IA: no domínio discreto, o BFS garante a solução de custo mínimo 5 e o algoritmo Guloso com poda de ciclos viabiliza a execução escalável até $L = 4096$; no domínio contínuo, o relaxamento fuzzy com gradiente descendente atinge o alvo em passo único para $\eta = 0{,}5$, enquanto a função custo contígua preserva a inversão em bloco respeitando a cota de estabilidade $\eta \leq \frac{1}{2+4\lambda}$.

Referências

[1] RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Pearson, 2020. Cap. 3: Solving Problems by Searching.

[2] SMOLA, A. et al. **Dive into Deep Learning**. Cambridge University Press, 2023. Cap. 12: Optimization Algorithms.

[3] PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. **NeurIPS 32**, p. 8024–8035, 2019.